

1교시: Docker Compose가 필요한 이유

수업 목표

- Day 3의 긴 docker run 옵션이 왜 handoff에 취약한지 설명한다.
- Compose가 service, network, volume을 하나의 project로 묶는 방식을 이해한다.
- docker compose config를 실행 전 검증 명령으로 사용한다.

50분 흐름

시간	활동	비중	산출
0-8분	Day 3 runtime option 복습	설명 15%	option list
8-18분	긴 명령의 재현성 한계 분석	설명 20%	risk note
18-30분	Compose application model 소개	설명 25%	service/network/volume map
30-42분	compose.yaml 위치와 config 확인	실행 25%	config evidence
42-50분	Day 4 실습 흐름 정리	설명 15%	handoff 기준

Visual 1: docker run에서 Compose로 이동

Compose로 실행 조건 파일화

```

$ docker run -d \
  --name my-app \
  -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e DB_HOST=db \
  -e DB_USER=app \
  -e DB_PASSWORD=secret \
  -v /data/app:/app/data \
  -v /data/logs:/app/logs \
  --network my-network \
  --restart unless-stopped \
  my-app:latest
        
```

```

compose.yaml
services:
  my-app:
    image: my-app:latest
    container_name: my-app
    restart: unless-stopped
    ports:
      - "8080:8080"
    environment:
      - SPRING_PROFILES_ACTIVE=prod
      - DB_HOST=db
      - DB_USER=app
      - DB_PASSWORD=secret
    volumes:
      - /data/app:/app/data
      - /data/logs:/app/logs
    networks:
      - my-network
        
```

이 visual은 port, environment, volume, network 옵션이 shell 명령에서 Compose file의 구조로 이동하는 흐름을 보여준다. 볼 지점은 "명령이 짧아졌다"가 아니라 "실행 조건이 파일로 남는다"는 점이다.

핵심 설명

Day 3에서는 web container, PostgreSQL container, network, volume을 각각 명령으로 만들었다. 이 방식은 학습에는 좋지만 인수인계에는 약하다. shell history에 남은 명령은 순서가 바뀌거나 일부 옵션이 빠지기 쉽고, 다른 사람이 같은 project 이름과 같은 port로 실행해야 하는지 알기 어렵다.

Compose는 여러 container를 하나의 application stack으로 정의한다. services에는 실행할 container의 image와 설정이 들어가고, networks에는 service 간 통신 boundary가 들어가며, volumes에는 container 삭제 후에도 남길 data 기준이 들어간다.

Compose를 쓰면 "web 실행", "DB 실행", "network 생성", "volume 생성"을 따로 외우는 대신 docker compose up -d로 project 단위 실행을 시작한다. 하지만 실행이 쉬워졌다고 검증이 사라지는 것은 아니다. Day 4에서는 config, ps, logs, run, down evidence를 반드시 남긴다.

실행 명령

```
cd week2/day4/labs/compose-app
cp .env.example .env
docker compose config
```

확인할 결과

docker compose config 결과에서 다음 항목이 보이면 기본 구조가 해석된 것이다.

```
services:
  db:
  web:
networks:
  app-net:
volumes:
  pgdata:
```

판단 기준

질문	확인 위치
외부에서 web에 접근할 port는 무엇인가	services.web.ports
DB 설정은 어디서 들어오는가	services.db.environment, .env
DB data는 어디에 남는가	volumes.pgdata
web과 db가 같은 network에 있는가	services.*.networks
실행 전 문법을 확인했는가	docker compose config

핵심 유의사항

Compose file은 실행 계약이다. YAML indentation이 맞아도 운영 의도가 틀리면 좋은 파일이 아니다. 예를 들어 secret을 그대로 박아 넣거나, DB volume 삭제 명령을 기본 cleanup처럼 기록하면 실습은 되더라도 운영 습관은 나빠진다.

docker compose config는 container를 만들지 않는다. 실행 전에 file을 해석하고 variable interpolation 결과를 보여주는 명령이다. Day 4에서는 문제가 생기면 up을 반복하기 전에 먼저 config를 본다.

기록 템플릿

```
## Lesson 1 Compose Motivation
- Day 3에서 Compose로 옮길 옵션:
- Compose file path:
- config result:
- 실행 전 발견한 문제:
- handoff에 좋아진 점:
```

마무리 점검

Compose는 여러 container의 실행 조건을 ____ 파일로 관리한다.
실행 전 YAML과 변수 해석은 ____ 명령으로 확인한다.

평가 기준

기준	2점 evidence
필요성	긴 docker run의 handoff risk를 설명했다
구조	service/network/volume을 구분했다
실행 전 검증	docker compose config 결과를 기록했다

공식 근거 링크

- Docker Compose: <https://docs.docker.com/compose/>
- How Compose works: <https://docs.docker.com/compose/intro/compose-application-model/>

선행 지식과 범위 경계

이 교시는 Day 3의 실행 옵션을 전제로 한다. 학생은 docker run -p, -e, -v, --network가 각각 port publishing, runtime configuration, storage mount, network membership을 다룬다는 것을 이미 경험했다.

오늘 처음 배우는 것은 Compose 자체보다 "실행 조건을 선언형 파일로 보존하는 방식"이다. Compose는 Dockerfile을 대체하지 않는다. Dockerfile은 image를 만드는 기준이고, Compose는 만들어진 image 또는 build context를 어떤 runtime 조건으로 함께 실행할지 정하는 기준이다.

Day 4의 범위는 로컬 개발과 실습용 multi-container 실행이다. production orchestration, auto scaling, rolling update, service mesh, cloud load balancer는 다루지 않는다. 이 범위 경계를 명확히 해야 Compose를 Kubernetes처럼 오해하지 않는다.

학술 기준 연결

ABET 관점에서 이 교시는 "문제 분석"과 "해결안 평가"에 해당한다. 문제는 긴 명령을 사람이 기억하고 재입력할 때 생기는 재현성 저하이고, 해결안은 실행 조건을 file-based specification으로 옮기는 것이다.

CS2023 competency 관점으로 보면 지식, 기술, 태도가 분리된다.

구분	Day 4 Lesson 1 적용
Knowledge	container runtime option과 application model의 차이
Skill	docker compose config로 실행 전 file 해석 결과 확인
Disposition	"내가 실행했다"가 아니라 "다른 사람이 재현할 수 있다"를 우선하는 태도

Bloom 단계는 기억보다 분석과 평가에 가깝다. 학생은 단순히 docker compose up을 실행하는 것이 아니라, Day 3 명령이 Compose 항목 중 어디로 이동했는지 분류하고 그 이유를 설명해야 한다.

실무에서 Compose가 쓰이는 위치

현업에서 Compose는 주로 다음 위치에 놓인다.

사용 위치	적합한 이유	한계
로컬 개발 환경	web, db, cache를 한 번에 실행	운영 장애 자동 복구는 약함
통합 테스트	CI에서 의존 서비스 준비	cloud-managed service와 차이가 있음
교육/실습	실행 조건을 명시적으로 보여줌	production 운영 모델로 과장하면 위험
작은 내부 도구	낮은 복잡도의 service 묶음	scaling, rollout, security policy는 별도 필요

Compose를 쓰면 개발자가 "DB는 따로 실행하세요"라고 말하는 대신 `compose.yaml`과 `.env.example`을 제공할 수 있다. 이것이 handoff 품질의 차이다.

실행 조건을 파일화한다는 뜻

실행 조건을 파일화한다는 것은 명령어를 예쁘게 접어 넣는 것이 아니다. 다음 질문에 대한 답을 repository 안에 남기는 것이다.

- 어떤 service가 실행되는가?
- 어떤 image 또는 build context를 쓰는가?
- host에서 접근할 port는 무엇인가?
- service 간 통신은 어떤 network에서 이루어지는가?
- runtime config는 어떤 변수 이름으로 주입되는가?
- data는 container 삭제 후 남아야 하는가?
- 정상 상태는 어떤 command와 output으로 확인하는가?
- cleanup은 container만 지우는가, volume까지 지우는가?

이 질문에 답하지 못하는 Compose file은 실행은 될 수 있지만 운영 문서로는 부족하다.

실습 확장: run 옵션 분해하기

아래 Day 3 명령을 보고 각 옵션을 Compose 위치에 적는다.

```
docker run -d \
  --name paperclip-day3-postgres \
  --network paperclip-day3-net \
  -e POSTGRES_PASSWORD=paperclip \
  -e POSTGRES_DB=paperclip \
  -v paperclip-day3-pgdata:/var/lib/postgresql/data \
  postgres:16-alpine
```

Docker run 조각	Compose 위치	운영 의미
--name	service name 또는 container_name	사람이 식별할 실행 단위
--network	networks	통신 boundary
-e POSTGRES_DB	environment	runtime configuration
-v pgdata:...	volumes	data persistence
postgres:16-alpine	image	실행 artifact

학생은 이 표를 채운 뒤 "어떤 항목은 Compose에 쓰지 않아도 되는가"를 토론한다. 예를 들어 container_name은 학습에는 직관적이지만 project 단위 scale이나 충돌 회피에는 service name을 쓰는 편이 낫다.

오해 점검 문항

아래 문항은 수업 중 빠르게 구두 또는 짧은 기록으로 확인한다.

문항	기대 답
Compose file이 있으면 Dockerfile이 필요 없는가	아니다. image build 기준과 runtime 실행 기준은 다르다
docker compose config는 container를 실행하는가	아니다. file 해석과 변수 치환을 확인한다
Compose는 Kubernetes와 같은 운영 플랫폼인가	아니다. 로컬/개발/테스트에 강한 application model이다
긴 명령을 README에 쓰면 Compose가 필요 없는가	작은 예제는 가능하지만 multi-service handoff에는 취약하다

실무 failure mode

Compose 도입 초기에 자주 생기는 문제는 기술 오류보다 문서 오류다.

Failure mode	실제 증상	예방
.env 안내 누락	새 사람이 config 단계에서 실패	.env.example과 required variable 명시
port 문서 누락	browser가 wrong port로 접속	README에 host port 기록
volume 위험 미기록	down -v로 data 삭제	cleanup 명령을 위험도별로 분리
service name 오해	host에서 db로 접속 시도	host 접근과 container DNS 구분
config 검증 생략	YAML indentation 오류를 늦게 발견	up 전 config를 기본 습관화

Evidence 수준 구분

수준	예시
약한 evidence	"Compose 실행됨"이라고만 기록
중간 evidence	<code>docker compose up -d</code> 명령만 기록
강한 evidence	config, ps, HTTP status, DB query, cleanup 결과까지 기록

현업 handoff에서는 강한 evidence가 필요하다. 특히 fresh clone을 받은 사람이 같은 결과를 만들 수 있어야 한다.

전이 과제

학생의 Week 1 미니 앱 또는 Day 2 static-site 실습을 기준으로 다음 질문에 답한다.

1. web service 이름은 무엇으로 둘 것인가? 2. host port는 충돌을 피하기 위해 몇 번으로 둘 것인가?
3. source file을 bind mount할 것인가, image에 COPY할 것인가? 4. DB가 없다면 Compose가 그래도 필요한가? 5. README에는 어떤 check 명령을 넣을 것인가?

이 과제는 Day 5 통합 실습에서 개인 프로젝트를 Dockerfile과 Compose로 정리할 때 다시 사용한다.

종료 전 한 줄 정리

Compose는 container를 쉽게 켜는 명령이 아니라, 여러 실행 조건을 팀이 검토하고 재현할 수 있는 파일로 고정하는 방식이다.

이 문장을 자기 말로 바꿔 쓸 수 있으면 Lesson 1의 핵심은 통과한 것이다.